

Interactive Beat Annotation GUI

-SoTA Deliverable-

Fernando Azeredo

Masters in Multimedia at Porto University

Supervisor: António Sá Pinto

February 2026

Index

List of Abbreviations.....	3
Introduction.....	4
Contextualization and Problem Framing.....	4
Research Questions.....	5
RQ1 – Current Beat Annotation Limitations.....	5
RQ2 – Proposed BA Novelties.....	5
RQ 2.1- Audio-to-Score allignment.....	6
RQ 2.2 – Spectrogram Audio Visualization in BA.....	6
RQ2.3 – SoTA BT Model Integration.....	6
RQ2.4 – Beat Quantization and Annotation Grid.....	7
RQ3 – Proving Value.....	7
State-of-the-Art.....	8
Beat tracking Research.....	8
Beat Annotation Methods.....	10
Existing Tools for Beat Annotation.....	10
HIL Approach for BT.....	12
Audio-to-Score Alignment for BT.....	13
Work Plan.....	14
1 st Iteration of the GUI.....	15
1 st Testing phase.....	17
Intermediate Presentation Preparation.....	18
2 nd Iteration of GUI.....	18
2 nd Phase Testing.....	21
Public Defense Preparation.....	21
Thesis Finalization.....	21
Appendix.....	22
Bibliography.....	23

List of Abbreviations

Abbreviation	Full Term	Description
GUI	Graphical User Interface	For the purpose of this deliverable, these terms are used interchangeably to refer to the: <i>“Digital Graphical User Interface For Beat Annotation”</i> as a whole.
UI	User Interface	Meaning all visual elements that will be displayed to the user visually. This is the main object of the thesis.
UX	User Experience	Term used frequently in tandem with UI – <i>“UI/UX”</i> . UI refers to: <i>“What the user sees.”</i> UX refers to: <i>“How the user interacts.”</i> UX relates to general workflow and experience of using the program.
MIR	Music Information Retrieval	Research field that develops computational methods to analyze, interpret, and retrieve musical information from audio.
BT	Beat Track/ing	The process of computationally and autonomously detect the temporal positions of beats or the underlying pulse structure within a musical audio signal
BA	Beat Annotation/ing	The act of <u>labeling precise temporal locations</u> of musical beats.
HIL	Human-in-the-Loop	A symbiotic interaction between the user and an automated system (ML). It adapts dynamically according to user input.
BPM	Beats Per Minute	A standard measure of musical tempo representing the <u>number of beats occurring within a one-minute</u> interval. It is also related to the concept of “tempo” . The latter being the musical term usually is more vague indicating a range of bpm (<i>andante, lento, largo, presto</i> etc.).
SV	Sonic Visualizer	The de facto standard open-source desktop <u>application used by MIR researchers</u> and audio/sound engineers for the <u>visualization and analysis of audio files</u> .
PP	Piano Precision	Open-source tool derived from SV’s source code, therefore it is considered an <u>SV Fork</u> .

Introduction

Contextualization and Problem Framing

Music Information Retrieval (MIR) is a research field that develops computational methods to analyze, interpret, and retrieve musical information from audio, examples include: harmony, melody, rhythm, and structural elements. Within MIR, Beat Tracking (BT) focuses on the autonomous extraction of musical beat information from recordings. Since the beat provides a fundamental temporal framework for music, BT is a critical component of many MIR systems.

“Many MIR systems first analyze the beats as the starting point, assume the results would be correct, and use them as a unit for further processing.” (Yamamoto, 2021)

The beat is typically defined as a regularly occurring pulse that underlies the temporal organization of a musical piece. However, the perception of “beat” can vary across listeners. An objectively definitive beat can only be established if it is explicitly defined by the composer or agreed upon by performing musicians.

BT has been a research focus for decades. Existing algorithms perform reliably well for music with steady, predictable tempos, such as pop music genres. In contrast, music with fluctuating tempos and non-percussive elements remain challenging for B. Examples of problematic music genres include: classical, jazz or world music. These can make the process of beat tracking more complex and ambiguous.

For this reason SoTA BT autonomous algorithms are all machine learning (ML) based. And although there are a few that are self-supervised (Desblancs et al., 2023), these are usually genre specific or are bound to certain musical configurations. For the cases that SoTA BT still can’t solve, human-made beat annotation (BA) are still the most reliable.

Although there has been a lot of effort in advancing autonomous BT solutions, the same can not be said for human focused BA tools and software, where it seems, they have remained largely unchanged for years now. The main difference nowadays, is that one can use a SoTA BT model to make a first assessment of the beat, however, for problematic BT music, this still implies a slow and cumbersome process for making that initial flawed BT into an accurate beat map. This process hasn’t changed much, at least for the last two decades.

Besides this, there is no standardized way for BA. Annotators end up developing custom ways of accomplishing the task, which might contribute to beat mappings with varying levels of accuracy. In big datasets made by multiple beat annotators, this may effect the overall BT model accuracy.

Research Questions

While there have been advancements in BT, the newer SoTA algorithms still struggle with expressive and non-percussive music. For cases where BT isn't reliable, human made BA is still required. There is also still the need for human made (or reviewed) annotations to train newer BT algorithms. Beyond that, BA technology has stagnated during the last decade, no new relevant BA graphical tool/software have been publicly released. The current de facto standard for BA, relies on the usage of stand-alone open-source desktop application: Sonic Visualizer (SV). While powerful and widely adopted by the MIR research community, this program has not updated its BA tools in years. The process of BA has stayed mostly the same for decades now. This creates a clear opportunity to advance beyond traditional-desktop based annotation workflows.

RQ1 – Current Beat Annotation Limitations

What limitations exist in current beat annotation workflows when applied to challenging BT music?

Annotating expressive music with high tempo variability is particularly "cumbersome," as annotators rely on manually tapping for adding beats, or drag&drop style methods to input/shift individual beats into their correct position. Tapping is a preferred method for some annotators, however, due to system latency, jitter and motor noise among other variables (Driedger et al., 2019) and failed taps, this method still requires manually going beat by beat making sure that they are as perfectly aligned as humanly possible.

RQ2 – Proposed BA Novelty

How can different musical, model and signal representations support and guide the beat annotation process?

To overcome hurdles, we propose a junction of various already existing musical and audio analysis and visualization tools. By integrating intelligent workflows, appropriate visualization tools, beat shifting operations and snapping features we intend to create a GUI that makes this process less cumbersome and more streamlined. Thus, we can divide this question into 4 concrete ones:

RQ 2.1- Audio-to-Score alignment

How can a symbolic score representation, defined in musical time, be aligned with an audio-based representation defined in absolute time?

Symbolic score information serves as a crucial reference for identifying metrical positions and expected note hits in classical or complex performances. Aligning “musical time” (which is not absolute, it is relative and it can vary) with the absolute audio timeline (seconds), should allow annotators to navigate and assess beat positions more accurately. This cross-modal alignment should also aid in resolving metrical ambiguities and ensuring that annotations remain consistent according to the score in question.

RQ 2.2 – Spectrogram Audio Visualization in BA

How can an audio spectrogram support accurate manual beat annotation in challenging music?

Spectrograms provide a vital visual anchor by revealing vertical spectral structures that originate from instrumental onsets. In challenging music, where the beat is not percussively explicit, these visual cues help annotators verify the exact temporal location of musical events (onsets). The spectrogram view fused with the proposed audio-to-score alignment may prove to be beneficial. Spectrogram for precise beat placement, Score visualization for onset-to-beat detection.

RQ2.3 – SoTA BT Model Integration

How can deep neural network output activations, representing the probability of beat presence over time, guide the annotation process?

SoTA BT and MIR research are mostly focusing on machine learning (ML) and deep neural networks (DNN) algorithms. How can these newer SoTA BT models aid on the task of BA? What about other MIR computational algorithms such as onset detection, can they be used for BA purposes? We believe that yes, they’ll play a crucial role in making BA less cumbersome. In some of the less “problematic” cases SoTA BT models can have a mostly correct BT, the most common flaws being: the half/double problem, false negatives (missed beats) and false positives (added beats) as well as slightly offset beats. The solution we envision for these problems evolves developing a GUI that encompasses different BA tools, some will benefit not only from SoTA BT models but also might benefit from other MIR ML and DNN models.

RQ2.4 – Beat Quantization and Annotation Grid

Can such activation functions be used to define or support a beat quantization grid during annotation?

This question acts as an extension of the previous one.

A concrete example would be: the use of MIR algorithms to generate a snap-to-grid like function. Where unaligned or incorrect beats snap to the next “most probable” position.

Such a feature could allow so: when a performing a beat tap (as previously established this, by it’s self, is a flawed method for BA), marked beats would not assume their absolute input time but would automatically “snap” into their most probable position. How can we design such system and can it prove useful for the BA process?

RQ3 – Proving Value

To what extent does the proposed beat annotation workflow affect annotation time and efficiency when compared with existing tools?

Ultimately this is the most essential question to be posed. The whole purpose of the development of this GUI, is for it to be a general improvement on decades old software and UI/UX workflows. Thus, the question of time and efficiency for the BA task is at the center of this thesis. However we believe that the features we propose already should aid on both these aspects, time and efficiency. By replacing individual drag-and-drop actions with a streamlined process of judging automatic generated recommendations, the system aims to dramatically increase annotation efficiency and decrease general duration of the task. The Score-to-Audio alignment should also introduce a new BA method that aims to streamline the process and aid in the making of more precise beat assessments.

State-of-the-Art

Beat tracking Research

It is clear that BT is one of MIR's most fundamental and most researched fields. As of now BT is considered a "solved" problem for music with never changing 4/4 metric and steady bpm. This includes the majority of western commercially available music (Country, Disco, HipHop, Rock, Metal, Pop etc.). Where SoTA BT models still falter is in music that falls from that standard of absolute and steady beat rhythmic organization.

Musical works with fluctuating bpm, complex or unconventional time signatures and non-percussive elements, SoTA BT models still falters. This includes music that is "acoustically" performed, meaning, music where the bpm can fluctuate for performative or expressive reasons. Examples include genres: Classical, Jazz and World Music.

Besides this, the act of assessing the beat can be ambiguous. In some examples, annotators might tap at half, double, or even triple the actual beat. It is also possible for one to feel the subdivision at different levels, and this may impact the beat assessment of the annotator. For these cases, there can only be one "correct" assessment if the recording in question has an underlying musical score that explicitly describes every beat, i.e. must classical music.

For these reasons, current SoTA beat tracking methods are predominantly based on neural networks. ZeroNS (Desblancs et al., 2023), BeatThis! (Foscarin et al., 2024), and HingeNet (Ru et al., 2025) are just 3 examples of SoTA algorithms that illustrate the complexity of the BT problem:

- ZeroNS (2023) was the first self-supervised BT model. It works by analyzing separately the percussive elements (drums) and melodic parts of a given recording. A problem with this model is that by design is dependent of instrumentation, this model works best for music with drums and clear melodic instrument distinction (voice, guitar, keys, bass etc...). Besides this, ZeroNS performs still performed poorly against other supervised SoTA models.
- BeatThis! Is still considered the SoTA BT model. It works by analyzing spectrogram windows of the recording. It's arguably the best BT model at the moment, and it works well for a vast amount of music genre.
- HingeNet combines different BT models and adds a harmonically aware layer on top. Although HingeNet is newer, F-measure tests show that it is slightly behind the BeatThis! model.

The performance of BT algorithms is based on an F-measure metric presented in 0.0 to 1.0 ratio or it's equivalent in percentage 0%-100%. This measures a BT model against a ground-truth given by a dataset. It takes into account correct beats, false positives (extra beats) and false negatives (missed

beats), it also allows for a $\pm 70\text{ms}$ window of tolerance for each beat. To understand the reason for this check: (Driedger et al., 2019).

BT models are tested against well documented datasets. In Figure 1 I present four that are common to see as benchmark tests for BT:

- GTZAN was specifically designed for musical genre classification. It is a collection of 2000 music excerpts of 30sec each. It covers 10 different music genres, plus 4 and 6 sub-genres of classical and jazz music respectively, each having 100 excerpts. This gives a total of 19h of audio to be tested.
- Ballroom is a dataset consisting a total of 698 total examples. With eight different dance music genres: Cha-Cha, Jive, Quickstep, Rumba, Samba, Tango, Viennese Waltz, and Slow Waltz. BPM ranges from 60-224. As this is dance-music, bpm stays mostly the same across the excerpts, for this reason it is expected for SoTA models to perform F-measures $> 90\%$.
- Hainsworth: A dataset consisting of 221 musical examples from a variety of genres, rock/pop, dance, classical, folk and jazz.
- SMC Mirex: Compiled to include pieces that are challenging for SoTA BT models. This is the benchmark test of BT algorithms. This data set consists of 217 manually-annotated and highly precise BA.

Dataset	Zero-Note Samba (2023)	Beat This! (2024)	HingeNet (2025)
GTZAN	87.5%	89.1%	89.7%
Ballroom	91.1%	97.5%	97.3%
Hainsworth	78.3%	91.9%	91.4%
SMC Mirex	52.0%	62.7%	61.7%

Table 1: Tabel comparing the performance (F-measure) of SoTA BT models.

(Desblancs et al., 2023; Foscari et al., 2024; Ru et al., 2025)

Beat Annotation Methods

For music in which autonomous SoTA BT systems aren't efficient enough, relaying on human made BA still might be preferable to the annotator:

“A common method to create beat annotations for music recordings is to let a human annotator tap along with them.”

(Driedger et al., 2019)

The problem with this, is that these annotations will probably not be accurate enough even if the user performing them is an experienced musician. There are multiple reasons that stem from perception, human motor noise, jitter and system latency (Driedger et al., 2019). Thus when tapping, there is the need to shift marked beats in order to make them accurate:

“This shifting operation is particularly relevant when correcting tapped beats, which can be subject to human motor noise as well as jitter and latency during acquisition.”

(Sá Pinto et al., 2020)

A tool that might prove useful for the shifting beats task is “Shift If You Can” (Sá Pinto et al., 2020). This is an open-source python library made specifically to aid in the development of such a task. In the [“2nd Iteration of GUI”](#) under the “Work Plan” sub-chapter, I describe in more detail how this shall be done and function.

Existing Tools for Beat Annotation

Sonic Visualizer is still the go-to software for scientific and MIR purposed audio analysis. However when it comes to BT, SV is outdated due to it's inefficient BT labeling process, that is still very much manual and cumbersome (Yamamoto, 2021). SV also is heavily reliant on VAMP plug-ins, modern SoTA BT algorithms are complex ML applications that may be difficult to implement into VAMP plug-ins and/or there is no direct or valued incentive for making these algorithms available in this way.

Time Instances inside SV (name of the main tool in SV for BT), can be exported and imported as simple .txt files. Thus, for one to benefit from a SoTA BT algorithm, one needs to follow a few steps:

- First, one needs to have access the algorithm in itself which might involve not only browsing/researching, but also probably, accessing some publicly available source-code repository.

- Secondly, one needs to know not only how to compile and run the program, but also export the retrieved data into a .txt file that SV can read.
- Besides this, we also need to take into account that to accomplish these steps, one needs at least, to be minimally literate/experienced with programming. Which might not be the case for a lot of beat annotators

A problem with this, is that other users, who may not be experienced in programming or literate in MIR research, will rely on the most easily accessible BT algorithm inside SV, and these will probably be BT algorithms such as: BeatRoot (S. Dixon, 2001), Tempo and Beat Tracker - Queen Mary University of London (Stark et al., 2009), and IBT – INESC Beat Tracker (Oliveira et al., 2010) among others. The reason being that, although they don't come pre-installed with SV, they can be added via a native feature that allows to browse and install VAMP plugins without exiting the program (referring to the “Find Transform” SV sub-menu). Crucially it is important to note that, as of the writing of this paper, you will not find any SoTA BT using this SV built-in feature, in fact one will only find outdated BT that are at least more than a decade old.

One of the most common autonomous BT assessment mistakes are the Double/Half time errors, in which the algorithm might mark the beat in orders of integer multiplication (or division) from the actually correct beat.

Another common mistake might be inducing multiple closely sequenced markings in the place where only one should be, or the contrary, omitting/skipping beats. For correcting these type of mistakes, current Beat Tracking software only allow correcting beats one-by-one. A simple workaround for this mistake would be to develop a feature in which the user could select a time region of beats and perform an operation such as:

‘In this region, that start at xx:xx and ends at yy:yy, the tempo is a steady X bpm with Y/Z timeSignature.’

The program would then shift/eliminate/highlight marked beats that deviate from the “user statement”. I specify in depth how this tool should function in the “[2nd Iteration of GUI](#)” under the “Work Plan” sub-chapter.

HIL Approach for BT

In 2021 Yamamoto proposed a Human-In-the-Loop (HIL) approach for BT, the problem is, although this paper seems promising, the software developed for this paper is closed-sourced, property of Yamaha Corp. and thus, it is not likely to be publicly accessible.

The paper also fails to mention how a spectrogram can be beneficial as opposed to the more conventional waveform representation of audio. A waveform can be useful for identifying general differences in energy, on the other hand spectrograms can, besides that, represent information related with spectral energy in time (frequency-energy).

A clear example of this would be: In a waveform, one can distinguish a *ff* section of from a *pp* in a recording. In a spectrogram, one can not only distinguish these, but also understand if a particular section would be chord hit, or an *arpeggio*, along side noticing where these note onsets would be located pitch wise.

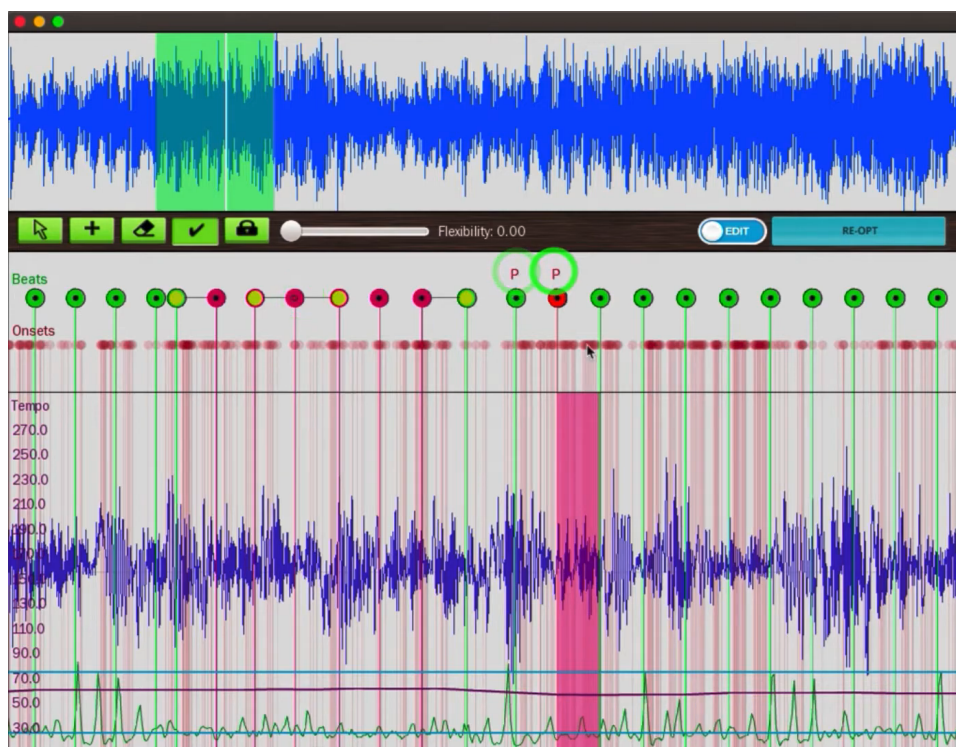


Figure 1: Yamamoto's HIL Approach for BT GUI. Screenshot taken from an official demo of the GUI in question. Link in [appendix](#).

Audio-to-Score Alignment for BT

Something that is not very well documented, is the relevance of following along a music score when performing BA. Besides helping the annotator follow along more accurately the recording, a score might notice the annotator in advance of any bpm or time signature changes. Besides that, when using a Spectrogram-Based view of the recording, one can distinctively align a specific onset to a concrete and exact moment in the recording timeline, this is essential to know if a beat is accurately assessed or not.

Piano Precision (Jiang et al., 2025), is an example of a SV fork that was designed for displaying musical scores alongside audio recordings in a unified graphical interface. This program works surprisingly well for classical Solo Piano pieces. Now there are two question that arrive from this:

- One is that, by following along the music sheet, Beat Tracking autonomously should be as straight forward as making the program align a beat marking with the respective note onsets that are marked in the score. Although Piano Precision already does this with high enough accuracy for Audio-to-Score alignment, for beat tracking adjustments might be needed, yet, it still might give amore accurate beat assessment than some SoTA BT.
- The other would be to integrate this idea of Audio-to-Score alignment with a Spectral View of the recording. In Piano Precision this is done via a side-window that displays the a single A4 page of the score, one at a time. When the user hits play, a darkened highlight displays the current playback position in the score, while in another window a spectral view of the audio is following along the playback position horizontally. This allows for a musically natural follow-along of the score, mimicking how a musical score is actually read and followed. However, for audio analysis and visualization purposes, it could be beneficial for the score to be aligned and viewed along side the playback position, meaning, horizontally and with note onsets that are aligned with their respective absolute position in the timeline. For BT this could allow the user to “grab a beat” and position it on the it’s corresponding position on top of the spectrogram representation. Also this could be beneficial for more easily detecting beats that should be shifted.

Work Plan

Table of Contents

Work Plan.....	12
1 st Iteration of the GUI.....	13
1 st Testing phase.....	15
Intermediate Presentation Preparation.....	15
2 nd Iteration of GUI.....	16
2 nd Phase Testing.....	18
Public Defense Preparation.....	18
Thesis Finalization.....	19

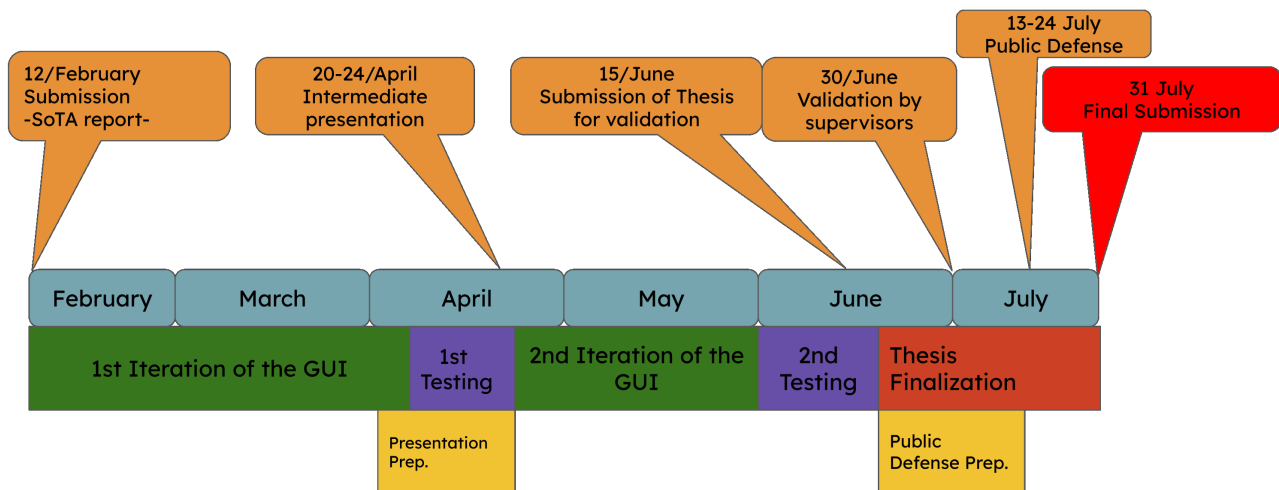


Figure 2: Timeline of planned deadlines for thesis developement.

Figure 2 shows a timeline with official deadlines and roughly marked self made deadlines. It is important to note that “thesis writing” is not inherently described in the timeline because it is implied that, the writing should be done along this whole period.

Each of the self imposed deadlines, marked in Figure 2 with the colored rectangles, have concrete objectives that should be met. These are described in detail in the following sub-chapters.

1st Iteration of the GUI

12th February — 5th April

List of main objectives:

- Decide: PP/SV fork VS [BeatMapJS](#) as starting point
- Basic barebones functional GUI
- Ability to upload a recording (.wav) with it's .mei score file.
- Basic BA features (tap beats, remove and adjust)
- Side-by-side (split horizontally) Spectrogram view of audio and score with roughly aligned score.
- View of note Onsets on top of Spectrogram (with show/hide function)
- Integrated SoTA BeatThis! BT algorithm. On top of the spectrogram (with show/hide function).
- Export beat mapping to .txt file compatible with SV.
- Score is set horizontally bellow the spectrogram. No notes need to be displayed. Just the measures with their counter (as described in the designated score): **1 measure should be the length between 2 downbeats.**

The focus of the **1st Iteration** is to have a functional barebones GUI that focuses on the Audio-to-Score alignment functionality. This is not only the most important feature for this work, but it also might be the most challenging to develop. The use of the source code for PP as a starting point might prove helpful, although might be difficult to implement due to various reasons. (see Table 2).

During the 1st term of this semester I developed a BA GUI with a Web based interface. Internally named [BeatMapJS \(link for repo. in Appendix chapter\)](#). This GUI was meant as a testing project, but it already has:

- Basic features for BA (beat tapping, remove, adjust, add labels and possibility to mark beats as downbeats)
- Working spectrogram view
- BeatThis! BT integration
- Export/Import BA to .txt compatible with SV.
- Other various UI/UX features (region marking and labelling, zoom in/out, spectrogram focused view, waveform view, beat labeling and assignable downbeat function)

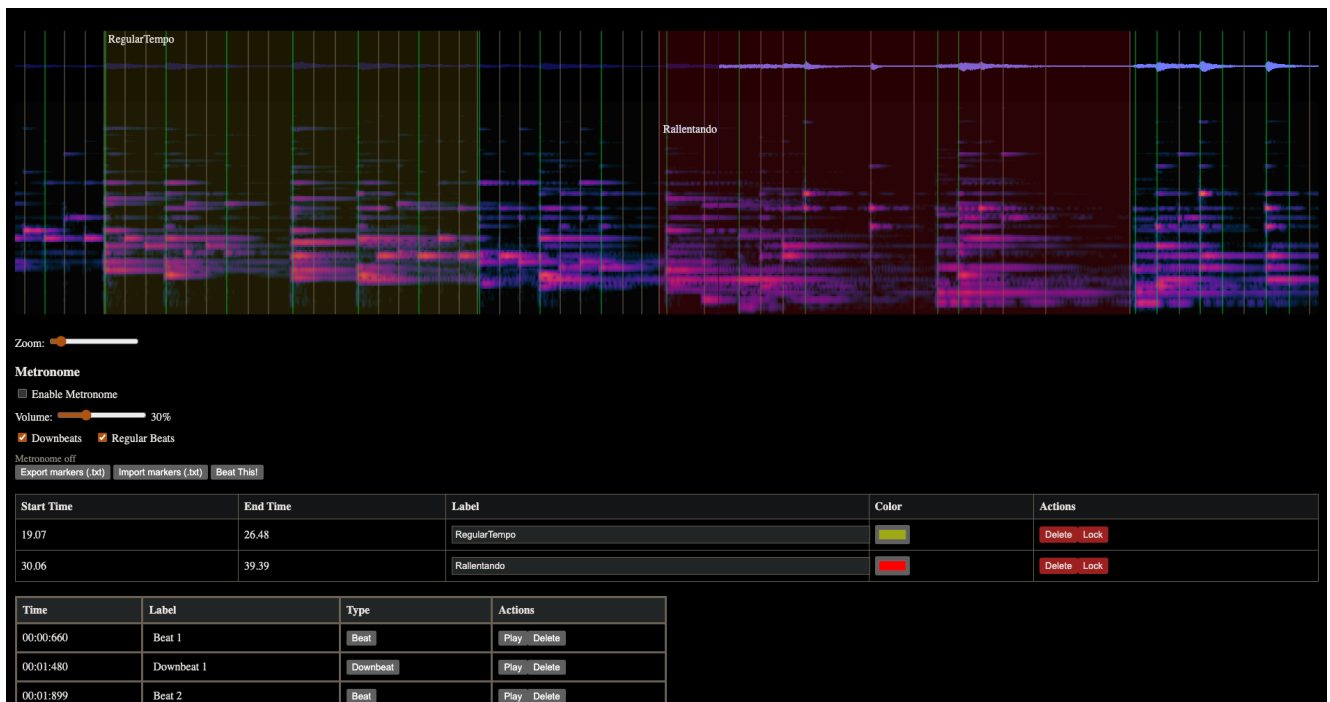


Figure 3: BeatMapJS interface, screenshot taken February 2026. GitHub repo. in Appenix.

First, it is important to decide if I should use BeatMapJS or Piano Precision (PP, an SV fork) as a starting point for the proposed GUI. Going the SV fork way, should help with the Audio-to-Score alignment and would probably lead into a more robust GUI due to SV's already established features. However, working with PP Source code might prove difficult because it is an extensive mainly C++/Qt code base, and honestly... I am not an experienced enough programmer, although not impossible... this makes me fear the worst :')

Sonic Visualizer Fork		Continuing BeatMapJS	
Cons	Pros	Cons	Pros
Large C++/Qt codebase	Polished visualization and playback Tools	Less polished UI/UX	Freedom to decide system architecture
Harder for quick prototyping	Established workflow for SV users	Non-existent initial user base	Faster and easier prototyping
Dealing with legacy code and design decisions.	Large and active user base.	Build audio & visual. Infrastructure from scratch	Ability to introduce modern technologies

Table 2: Example Pros&Cons of using SV's (or PP's) codebase against continuing to develop my own BeatMapJS.

On the other hand, BeatMapJS is a JavaScript/HTML/python codebase fully developed by me. Although still clunky, it already has essential built-in features. The three main discussion points for reeling on my BeatMapJS repo. are:

1. I would be developing a new web-based GUI (this is not necessarily bad, just a statement with it's caviats)
2. Not tacking advantage of SV's and PP already robust codebase and established UI elements.
3. Having to build the score alignment feature from the ground up.

For making the score-to-audio alignment I mean to remove every notes and simply represent the score with time signatures and empty measurements (with simple beat markers on top). The reason being that, displaying notes and rhythms on top of the score opens up a to a myriad of interface problems that I describe in more detail in the "[2nd Iteration Phase](#)" chapter. The importance here is to **lay the foundation for the audio-to-score alignment for BA**. There is an open source tool that might prove helpful in this task, this is the "ScoreWarp" repository (Goebel, 2025). As described in the [GitHub repo](#):

"ScoreWarp is a JavaScript package that warps a score (...) in order to horizontally align its elements with a given physical performance timeline."

It is important to note that, during this phase, if every objective is met before the self-imposed deadline, I should try to implement 2nd phase development tasks. The more complete the proposed GUI is in this phase, the better.

1st Testing phase

6th of April — 20th of April

List of essential objectives:

- Show developed UI to experienced annotators and other relevant personal.
- List possible UI/UX improvements/changes

In this phase I should be able to have an "alpha" version of the proposed GUI. Although being a barebones showcase, it should be in such a state where it is possible for selected users to test it in a controlled way. Meaning, with a selected and pre-made session (i.e. chosen music with already aligned score and BA made), users should be able to alter variables add labels and adjust beats etc. The idea here is not for testers to develop a full BA "from scratch" but to function more as a showcase of the proposed UI/UX. The focus of this testing phase should be to: serve as proof of concept that Score-to-Audio alignment and spectrogram are useful tools for BA.

Intermediate Presentation Preparation

1st April — 20th April

List of essential objectives:

- Prepare slide presentation
- Rehearse presentation
- Record/Present Demo of developed GUI

It is essential to prepare a good presentation. The focus should be on providing value proof of the tools I will be developing. It is also essential that the presentation has a working demo of the proposed GUI, a video recording of the GUI in action should be enough, although live demo would be better.

2nd Iteration of GUI

24th April — 1st June

List of main objectives

- Develop/Implement essential features listed in the 1st Testing Phase.
- Snap to beat functions. Develop user choices: *snap to onset, snap to beat, no snapping*
- Show complete score aligned with the spectrogram (user shall need to adjust portions according to the BA, but that feature should have been already implemented as stated on the 1st phase of development).
- Show problematic beat regions, these should be regions where the beat assessment is irregular or unconventional. User should be able to cycle through these sections quickly in order to flag them as correct or incorrect. The user can then (or later) make the necessary adjustments in problematic flagged regions.
- Develop Region Functions: Delete/Add/Edit multiple beats in a given user defined region. This should serve as a function to among other things, manually correct the half/double BT problem.
- UI/UX various improvements (look and feel, keyboard shortcuts etc.)

If on the 1st phase of development the focus was on introducing a novel BA feature, the Score-to-Audio alignment. On the 2nd phase of development I should be focusing on adding features that actually improve on current BA methods. Among various improvements three stand out:

1. Snap beat feature to align incorrect beats either to onsets or beats assessed by BeatThis!. This should be an on/off function. Can be helpful to enable a more accurate, manually tapped, BA or even to shift BeatThis! beat assessments to close onsets.
2. Add a GUI tool to solve “half/double time” BT assessments.
3. Full Score-to-Audio alignment. On the first stage, I focus on making the score align by making each measure correspond to their real time length. This should be the ground work to then implement the a full score visualization with notes. Although a lot of questions will come from this like:
 - If the score has multiple instruments (ex. orchestra, choral, ensamble etc.) how will it the representation work?
 - How will view zoom in/out work with the score allignement? When zooming out, notes and measure can get all scrambled. Do we define the zoom to be limited in a given range for this not to happen?
 - If there is a staff or time signature change on the beginning of the measure how should the alignment be made for the measure vs first note of the mesure?

The first improvement point (snap beat feature), is one that will benefit from the python library “Shift If You Can” developed by the thesis supervisor (Sá Pinto et al., 2020). This repository is a library for beat shifting related operations. This might be especially helpful for this task, but might also be useful in other aspects of the project (such as the labelling os “problematic” areas).

The second point is one clear improvement on current BA software (SV and Audacity). This is the most common BT error for decades now being commonly referenced in a lot of BT literature: (Dixon, 2001), (Gouyon et al., 2006) ,(González, 2013), (Foscarin et al., 2024). The solution I envision for this is to let the user define the given region that is affected by this, it should function as such:

- Set the bpm of that given region and/or set if the assessment is half/double/triplet. Beats that are off will be highlighted so the user can then confirm the alteration. So If a user defines:

“This section is marking double time starting at x beat of x measure and goes back to normal on y beat of y mesaure.”

If this was the case, then every other beat marking will be eliminated, starting at beat x measure x until beat y measure y.

This would quickly solve the section while at the same time correcting the measurement and beat counting label.

In addition to this, the user could set if the bpm of that section is regular or if it fluctuates. Meaning beats of that section could either: Be set with a regular interval between them (as set by absolute bpm value). Or if this section has fluctuations in bpm, then the beat would snap to the closest onset or beat prediction.

If there is a section, where the BT seems to be almost correct but with a few false positives (added beats) and/or false negatives (omitted beats) and/or slightly shifted beats every now and again. To correct this, as in the previous example, the user could set if the bpm of this region so that beats either are forced to be equally distanced (as set by user bpm) or snapped into closest onset/beat prediction.

It is also important to note that these features are supposed function in a “multiple” to “group” to “individual” workflow in mind. Where the user corrects a whole section, if he deems-it correct then he can move on, if there if most beats have been correctly assessed but a few were incorrectly snapped, than the user should be able to assess those beats individually or in tandem by either trying a different snap function (snap to onset instead of beat prediction) or if needed manually adjust via our good old friend click&drag.

For the third point (full score-to-audio), which might prove one of the most challenging features to implement I still don't know how to tackle. I believe I first have to accomplish the 1st Iteration and go through the 1st phase of testing to define better what I need to achieve in this topic.

One thing is clear in my mind. The adoption of a segmented BA process UX wise. By this I mean that any change needs to be confirmed by the user in order to be “locked” in the beat assessment. Changes are made in section that are either automatically or manually designated, within these sections, the original/previous states are always shown as a “un-highlighted/darkened” beat markings, until the user submits the given section as “corrected”. The benefit is that one, can always have the previous state as visual comparison against the new markings. This way if the user has a mental mapping of where multiple beats should be than he can more easily assess which tool to use and if the automated correction is ok. Also in a given section, the user could cycle through beat snapping, onset snapping, grid snapping etc, while still having the visual cue of the previous state. The user might also find helpful the ability to quickly switch between hearing the old version, and the new possibility. This also turns BA a more systematic process where the user has to complete x amount of tasks thus making BA a more standardized process.

2nd Phase Testing

1st June - 13th June

During this phase thesis doc. should be 85% written by it's end 100% (before revision phase).

List of main objectives:

- Final testing phase of the proposed GUI
- Utilizing testers feedback to make conclusions for thesis
- Listing flaws reported by testers

For this phase to start my GUI needs to be accomplished, I shall not change or add anything in the source code beyond this point. This is the essential phase to deliver a complete thesis report. It is through this phase that I will assume conclusions of the effectiveness of my proposed GUI and it is through it that I will objectively understand my project's strengths and flaw. Besides, this is an essential step to know how a work like this can move forward and contribute to MIR and BT research fields.

Public Defense Preparation

15th June — 24th July

List of essential objectives:

- Prepare slide presentation
- Rehearse Public Defense
- Record/Present Demo of developed GUI

This is it. The final presentation of my work. The place where I can explain in face to face my thesis work, it's relevance and it's impact. It is fundamental that I have a well prepared and well rehearsed Public Defense.

Thesis Finalization

15th June – 31st July

List of main objectives:

- Finalize thesis document with supervisor's advised corrections

- Formatting thesis details
- Review whole bibliography
- Prepare project repository for publishing

By the 15th of June (Submission of thesis for validation) a complete thesis should be submitted. Of course there will probably still be points to add or polish but those should come from my supervisor and other academics in the field.

I should take advantage during time, to be sure that my thesis is a polished and complete work.

If we intend for my work to be publicly published, then I shall use this time to make sure that my source-code is well documented, so that future work (either by myself or others) can be facilitated.

Appendix

Link “Yamamoto’s HIL GUI for BT” youtube demo:
<https://www.youtube.com/watch?v=euBbBltgEbM>

Link for BeatMapJS repo:
<https://github.com/FernandoAze/BeatMapJS.git>

Link for ScoreWarp repo:
<https://github.com/iwk-digital/scorewarp>

Link for ShiftIfYouCan repo:
<https://github.com/asapsmc/ShiftIfYouCan>

Bibliography

- Desblancs, D., Lostanlen, V., & Hennequin, R. (2023). Zero-Note Samba: Self-Supervised Beat Tracking. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31, 2922–2934.
- Dixon, S. (2001). *An Empirical Comparison of Tempo Trackers*.
- Dixon, S. (2001). An interactive beat tracking and visualisation system. *Proceedings of the International Computer Music Conference (ICMC)*, 215–218.
- Driedger, J., Schreiber, H., de Haas, W. B., & Müller, M. (2019). Towards Automatically Correcting Tapped Beat Annotations for Music Recordings. *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*.
- Foscarin, F., Schlüter, J., & Widmer, G. (2024). Beat this! Accurate beat tracking without DBN postprocessing. *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR 2024)*.
- Goebl, W. (2025). *Let's do the ScoreWarp again! Shifting notes to performance timelines*.
- González, J. R. Z. (2013). *Comparative Evaluation and Combination of Automatic Rhythm Description Systems*.
- Gouyon, F., Klapuri, A., Dixon, S., Alonso, M., Tzanetakis, G., Uhle, C., & Cano, P. (2006). An experimental comparison of audio tempo induction algorithms. *IEEE Transactions on Audio, Speech, and Language Processing*, 14(5), 1832–1844.
<https://doi.org/10.1109/TSA.2005.858509>
- Jiang, Y., Weigl, D. M., & Goebl, W. (2025). Piano Precision: Advancing Music Performance Analysis by Integrating User-Correctable Audio-to-Score Alignment. *Proceedings of the International Computer Music Conference (ICMC)*.
- Oliveira, J. L., Gouyon, F., Martins, L. G., & Reis, L. P. (2010). IBT: A Real-Time Tempo and Beat Tracking System. *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*.
- Ru, G., Wang, J., Zhao, J., Wu, Y., Yu, Y., Jiang, N., Wang, W., & Li, W. (2025). *HingeNet: A Harmonic-Aware Fine-Tuning Approach for Beat Tracking* (arXiv:2508.09788). arXiv.

<https://doi.org/10.48550/arXiv.2508.09788>

- Sá Pinto, A., Domingues, I., & Davies, M. E. P. (2020). Shift If You Can: Counting and Visualising Correction Operations for Beat Tracking Evaluation. *Extended Abstracts for the Late-Breaking Demo Session of the 21st International Society for Music Information Retrieval Conference (ISMIR)*.
- Stark, A. M., Davies, M. E. P., & Plumbley, M. D. (2009). Real-time beat-synchronous analysis of musical audio. *Proceedings of the 12th International Conference on Digital Audio Effects (DAFx-09)*.
- Yamamoto, K. (2021). Human-in-the-Loop Adaptation for Interactive Musical Beat Tracking. *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*.